

A Comparison of Three Open-source File Indexers: Robinhood, GUFi, XDU

Robert Bjornson
Yale University
New Haven, CT, USA
robert.bjornson@yale.edu

Abstract

Filesystems for high performance computing clusters have grown tremendously. It is not unusual for HPC clusters to store several billion files. This makes managing and reporting on file system usage nearly impossible using traditional Unix tools such as `find` and `du`. Such queries can take hours or days to complete on very large filesystems, and can impose a heavy load on metadata service. An attractive alternative is to build a metadata database alongside the storage system proper. If implemented in local flash memory and powered by modern multicore CPUs, queries can be greatly accelerated, while removing load from the storage system. In addition, the possibility exists for richer, Relational Database (RDB) queries that are more powerful than those possible via `find`. This paper compares three open-source indexers to a commercial product (Starfish) and to `find` and `du`.

Keywords

file system indexing, HPC storage, parallel scanning, storage analytics, Robinhood, GUFi, XDU, Starfish

ACM Reference Format:

Robert Bjornson. 2026. A Comparison of Three Open-source File Indexers: Robinhood, GUFi, XDU. In *Practice and Experience in Advanced Research Computing (PEARC '26)*, July 26–30, 2026, Minneapolis, MN, USA. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3785462.3815846>

1 Introduction

HPC storage systems typically store billions of files. For example, Yale's HPC storage currently contains over 5.2 billion files. Managing and reporting on such a huge set of files is both critical and very difficult using existing Unix tools. Maintaining a separate, queryable index of all files can greatly aid in managing large collections of files by offloading searches and providing a richer query language.

One commercial file indexer, Starfish [9] has developed a strong following among HPC centers, including at Yale. It can handle billions of files with ease, and provides a rich query API as well as GUI reporting tools. Beyond simple reporting and querying, it also has actions built in, such as automated deletion, migration and archiving. However, Starfish is licensed by capacity and at the multi-petabyte scale can be prohibitively expensive for many academic HPC centers.

In this paper we explore three open-source alternatives: Robinhood, GUFi, and XDU. Each has particular strengths and weaknesses, and none is quite so full-featured as Starfish. However, for many HPC centers they may well suffice.

1.1 Robinhood

Robinhood [2] [5] is similar to Starfish in using a MariaDB relational database to store its index. This database should be run on a capable database server with directly attached NVME storage. Each filesystem scanned is given a separate database. The database structure is straightforward; NAMES and ENTRIES tables describe the individual files and directories, while ACCT_STAT has rolled up stats for users and groups. These stats can be used to quickly generate a number of predefined reports: usage by group, user; largest files; oldest files, etc. Conspicuously, Robinhood does not by default provide rolled up file count and size information by directory, so `du` and counting files of large subtrees is somewhat slower than the other tools we investigated. On the other hand, Robinhood's `find` has a rich set of filters. See Figure 2b.

1.2 GUFi

GUFi [8] [7] takes a very different approach. A key design goal was to allow any user to query the filesystem, while only allowing that user to see information about files that they can access. This is in contrast to the other tools, which allow metadata on all files to be queried. GUFi implements the index by building a replica of the source tree in file storage. Each source directory is replicated with the same permissions as the original. The files in each directory are replaced by a single SQLite [4] file that contains metadata about all files in that directory. Typically the replica is built on a local NVME drive. The total size of the replica is much smaller than the original, albeit not as small as XDU's replica. Queries that involve multiple SQLite files can be processed in parallel, and typically, the `du` and `find` equivalents require accessing many SQLite files. However, GUFi has an optional post-indexing rollup that starts at the leaves of the tree and rolls up information to the parent SQLite files recursively. This allows many queries to be satisfied via a single SQLite file. The rollup increases the storage requirement of the index by duplicating information. To avoid creating huge SQLite files at the higher levels of the tree, this process is 'cut off' after a certain number of files is reached, which is configurable. All queries transparently limit the user to those files and directories that they have permission to access.

1.3 XDU

XDU [6] is similar to GUFi in using files stored in a local NVME drive, rather than a relational database. However, in this case, the



This work is licensed under a Creative Commons Attribution 4.0 International License. PEARC '26, Minneapolis, MN, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2377-3/26/07

<https://doi.org/10.1145/3785462.3815846>

data is stored in a set of parquet [1] files. The root directory and its direct subdirectories are replicated, and one or more parquet files are created in each subdirectory. Assuming that the root has a large number of immediate subdirectories (e.g. /home or /project), and a large number of parquet files per subdirectory, this permits a large degree of parallelism for queries. XDU's find has a much more limited set of filters than the other tools.

1.4 Comments

It should be noted that all of these tools require the scanner to run as an administrator (root), unless there is another account that has read access to the entire file tree. The tools differ in what access is required for the query. For Robinhood, the query needs access to the database. For XDU, read access to the parquet files. For GUFi, by design, the files and directories in the replica are the same as the original; any user can access their own metadata. However, in order for an administrator to make general queries, those queries will be run as root. Alternatively, GUFi's index files can be written using a setfacl default on the top level directory, automatically giving an admin or admin group read/execute access.

In the remainder of the work, we provide benchmarks on indexing and querying two different file trees, using a number of sample queries of interest to both users and system administrators.

2 Experimental details

The following versions were used: Starfish 6.7.12877+13514a036b; Robinhood 3.2.0; GUFi 0.6.10; XDU 0.4.1.

Two different sets of files were used in testing indexing and querying.

Benchmark set 1: static set of 10.3M files spread across 1.14M directories. Stored on SSD VAST system.

Benchmark set 2: live working tree of 116M files across 2.77M directories. Stored on Lenovo GPFS.

Two cluster nodes were used for benchmarking: dual 8358 CPUs @ 2.60GHz with 1 TiB of RAM and 6.9 TB NVME drive. For Robinhood, a containerized mariaDB server was utilized. Starfish results were obtained using Yales' production Starfish servers.

3 Performance Benchmarking

3.1 Indexing

Figure 1a shows the time required to index each of the datasets by each tool. Figure 1b shows size of the resulting index. Times shown are per 1M files. For Starfish, we estimated the index footprint by determining the total size of our Starfish index, and scaled it by the size of our benchmark set. For Robinhood's rollup, the cutoff was 10000.

Indexing times for the various tools varied less than the storage requirement. Index time is largely determined by the file system performance and the degree of effective parallelism of the scan. In contrast, the index size varied tremendously, ranging from MiBs (XDU) to GiBs (GUFi with rollup) per million files. In particular, XDU's use of parquet results in very small index files.

Indexing of 1 billion files would likely take on the order of a day with these tools, which seems feasible. The resulting indices are of a manageable size, with the possible exception of GUFi with rollup. That size may be due to our choice of cutoff.

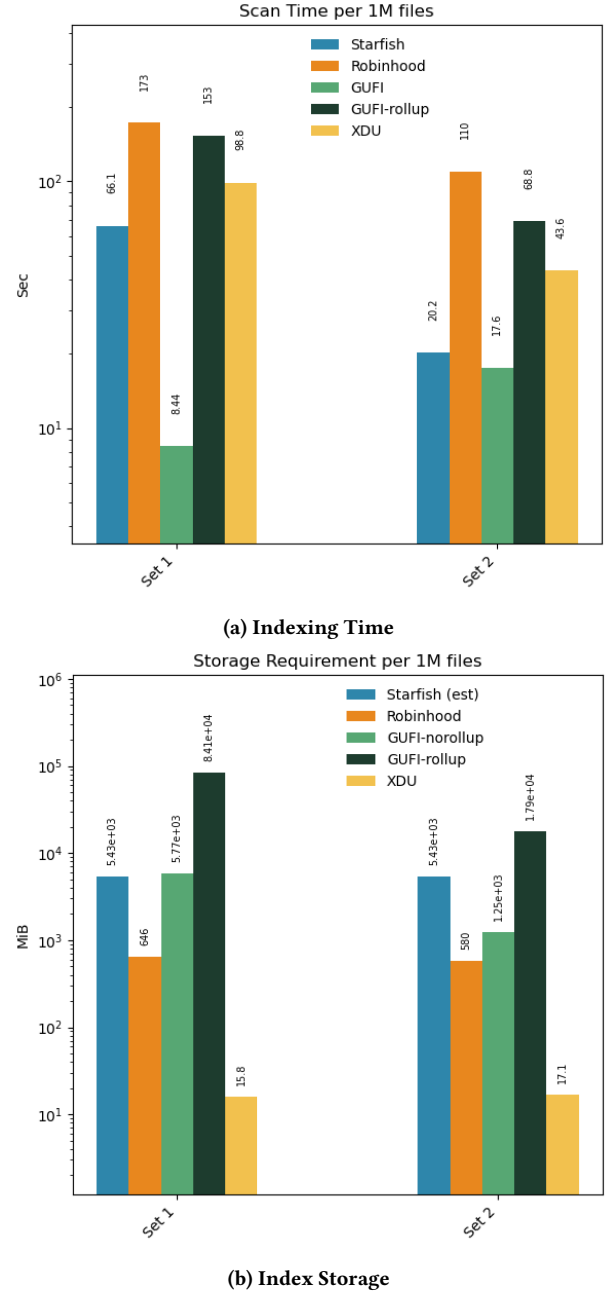


Figure 1: Tool Scan Time and Storage Requirement

3.2 Queries

We tested the tools using five representative queries, see Table 1. The results are presented in Figure 3 on a log scale. Each query was replicated three times, and one standard deviation is shown. Most of the variations were small. However, the first query in a series was often slower for Robinhood, GUFi, and XDU. We attribute this to initial data loading into memory. We also note that for some tests, the find and du queries were actually faster on the larger fileset.

```

rdb9@r813u09n02.mccleary:~
bc447
. . .
. .conda 112.19 GiB 711.6K files 14 days ago
. .conda_rsync 13.56 GiB 206.9K files 14 days ago
. .conda_tar 13.56 GiB 206.9K files 14 days ago
. .ollama 1.50 KiB 3 files 14 days ago
. .vim 1.22 MiB 80 files 14 days ago
. R 6.77 MiB 282 files 14 days ago
. bin 32.71 MiB 3 files 14 days ago
. conda_envs 564.14 MiB 10.8K files 14 days ago
. conda_pkgs 4.45 GiB 66.6K files 14 days ago
. llm 201.50 KiB 190 files 14 days ago
. ollama 3.12 GiB 22 files 14 days ago
. testbrowser 131.50 KiB 75 files 14 days ago
. tickets 1.21 MiB 648 files 14 days ago
. tools 325.00 KiB 41 files 14 days ago
. workshop 138.00 KiB 52 files 14 days ago
. ycrc 64.73 MiB 372 files 14 days ago
. ycrc-ebfiles 103.55 MiB 3.6K files 14 days ago
. vimrc 1.00 KiB 1 file 14 days ago

```

(a) xdu-view tool

	find	Starfish	Robinhood	GUFi	XDU
min/max depth	●	●	●	●	●
name regex	●	●	●	●	●
min/max size	●	●	●	●	●
access time	●	●	●	●	●
create time	●	●	●	●	●
modify time	●	●	●	●	●
group	●	●	●	●	●
owner	●	●	●	●	●
mode	●	●	●	●	●
type	●	●	●	●	●
negation	●	●	●	●	●

● Supported
● Not supported

(b) Feature comparison

Figure 2

This may be due to the two filesets being stored on different storage technology.

XDU does not provide a simple replacement for `du`. Instead, an ascii-based file browser is provided, `xdu-view`, which instantly displays the file count and total size of each directory as the user traverses the file tree. See Figure 2a.

All of the tools were orders of magnitude faster than `find` and `du`. Many of the queries were well under one second. XDU and GUFi with rollup generally performed the best.

Indexing time, index size, and query time are not the only relevant criteria for tool choice. It is critical that the tool support the queries that the user requires. The tools vary quite widely in the filters available for the search command. Figure 2b compares the predicates supported by each tool. Most support most or all of the important `find` predicates. XDU is notable for having a relatively restricted set.

- Q1 find one unique file by name
- Q2 find all files matching `slurm-*.out`
- Q3 find all files size > 500MB
- Q4 disk usage of large subdirectory
- Q5 count files in large subdirectory

Table 1: Queries

4 Using Richer Relational Queries

All three open-source tools permit some degree of direct relational queries.

4.0.1 Robinhood. Since Robinhood's index is an RDB, it is available for direct query in the usual way.

4.0.2 GUFi. GUFi's index is distributed across a huge number of SQLite files, one per directory. In fact, GUFi's `gufi-find` and `gufi-du` are actually large parallel queries of the SQLite files, and in principle one can build one's own queries. The documentation cautions that this is an "advanced" feature. We did not investigate this further.

4.0.3 XDU. XDU's parquet files can be queried directly using DuckDB [3]. However, since the files contain only 3 items per entry: path, size, and atime, the power of DuckDB queries is mostly limited to what can already be achieved via `xdu-find`. Duckdb can be useful for 'group by' queries or importing data into python, e.g. as a dataframe.

5 Conclusions

Managing a multi-petabyte storage system with billions of files requires better tools than `du` and `find`. Some filesystems (e.g. VAST) have a built-in index. However, these tools are typically not available to users. In addition, many HPC centers have multiple storage vendors and can benefit from a universal tool.

The open source tools we tested are not as full-featured as Starfish, but for many centers they may be good enough. All of the tools are orders of magnitude faster than `du` and `find`. Robinhood has a rich predicate set, a modest footprint and reasonably fast queries. It would benefit from a rollup in order to better support `du` and file counts. It also requires supporting a database server. GUFi supports a large set of predicates, is fast, and permits non-privileged users to search and report on their files, while preventing them from reporting on files they cannot access. XDU combines a remarkably small footprint with fast query times, but suffers from having fewer predicates. GUFi and XDU have the advantage of not requiring a separate relational database server.

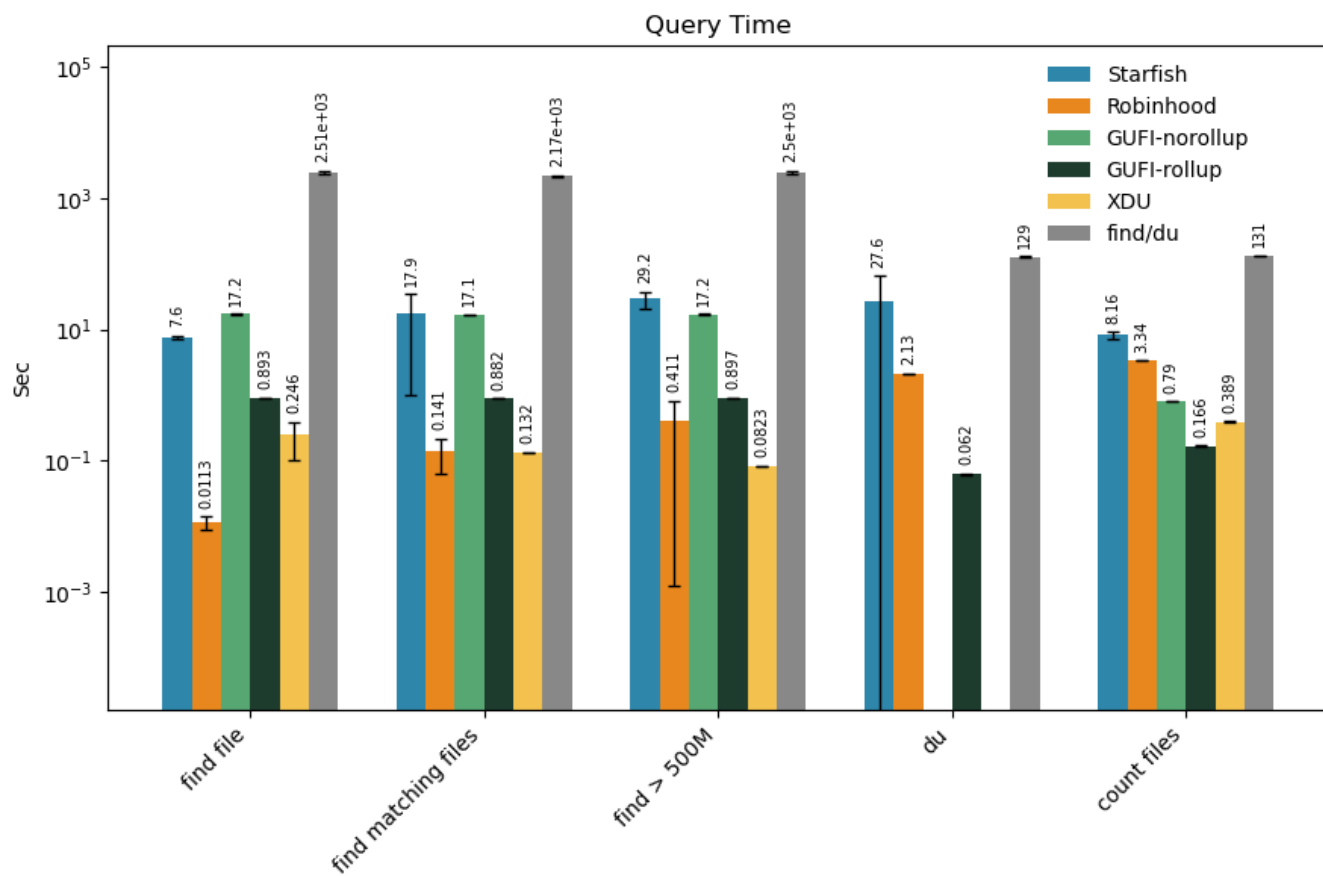
Since these projects are open source, it should be possible to extend them, for example adding additional metadata fields to XDU or a rollup to Robinhood. We hope to investigate these ideas further.

Acknowledgments

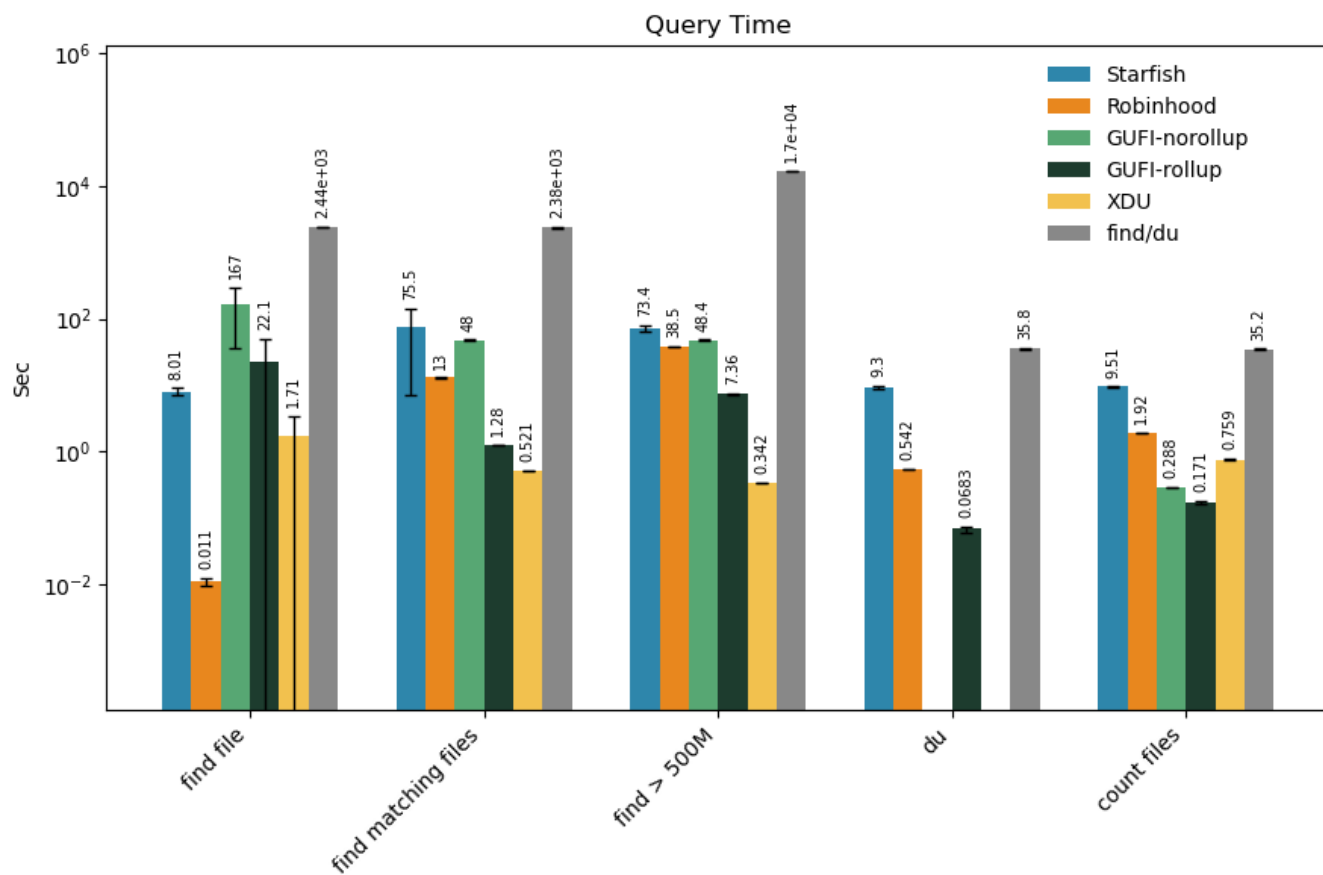
Thanks to Tom Langford of YCRC for his invaluable feedback on this project and manuscript.

References

- [1] Apache Software Foundation. 2013. Apache Parquet. <https://parquet.apache.org>. Accessed: 2026-03-20.



(a) Queries on Set 1 (10.2M files)



(b) Queries on Set 2 (116M files)

- [2] CEA/DAM. 2024. Robinhood Policy Engine: A Versatile Tool to Monitor Filesystem Contents and Schedule Actions on Filesystem Entries. <https://github.com/cea-hpc/robinhood>. Version 3.2.0. Accessed: 2026-03-05.
- [3] DuckDB. 2026. DuckDB is a fast portable database system. <https://duckdb.org/>.
- [4] D. Richard Hipp, Dan Kennedy, and Joe Mistachkin. 2020. *SQLite*. SQLite Consortium. <https://www.sqlite.org>.
- [5] Thomas Leibovici. 2015. Taking back control of HPC file systems with Robinhood Policy Engine. *arXiv preprint arXiv:1505.01448* (2015).
- [6] Geoffrey Lenter. [n. d.]. Extreme-scale parallel "du" command. <https://github.com/glentner/xdu>. Version 0.4.1.
- [7] Dominic Manno. [n. d.]. Grand Unified File-Index. <https://github.com/mar-file-system/GUFI>. Version 0.6.10.
- [8] Dominic Manno, Jason Lee, Prajwal Challa, Qing Zheng, David Bonnie, Gary Grider, and Bradley Settlemyer. 2022. GUFI: Fast, Secure File System Metadata Search for Both Privileged and Unprivileged Users. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–14. doi:10.1109/SC41404.2022.00062
- [9] Starfish Storage Corporation. 2026. Starfish: Unstructured Data Management and Metadata for Files and Objects. <https://starfishstorage.com>. Accessed: 2026-03-05.